

Project 2: Image Generation

1. 모델 아키텍처

공통 핵심 블록

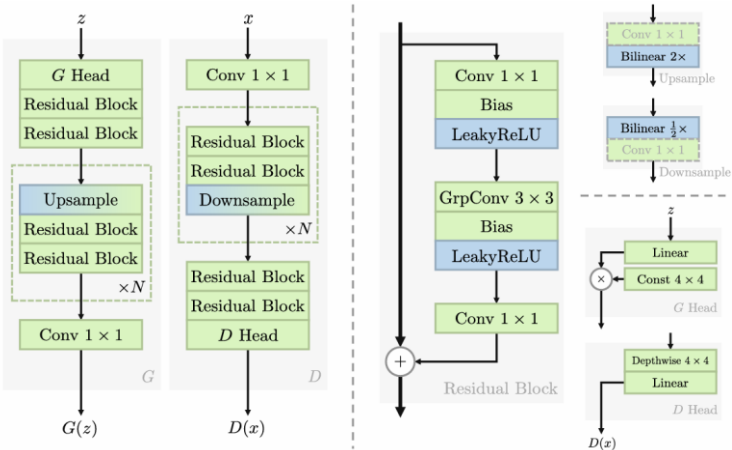
R3ResidualBlock	ResNeXt-style residual block. 1×1 Conv $\rightarrow$ grouped 3×3 Conv $\rightarrow$ 1×1 Conv 구조로, 중간 convolution 에서 stage 별 cardinality 를 사용한다.
BiasLeakyReLU	채널별 learnable bias 를 더한 뒤 LeakyReLU 를 적용하는 activation. Exact GP 계산 시 activation mask 를 기록하는 기준이 된다.
FixedFilterUpsample	고정 (1, 2, 1) FIR-style filter 를 사용하는 $\times 2$ upsampling layer.
FixedFilterDownsample	고정 (1, 2, 1) FIR-style filter 를 사용하는 $\times 2$ downsampling layer.

Generator 아키텍처 (z (B,512) $\rightarrow$ **GenerativeBasis** $\rightarrow$ **Progressive Stages** $\rightarrow$ **to_rgb** $\rightarrow$ **Image**)

GenerativeBasis	512 차원 latent vector z 를 4×4 feature map 으로 변환하는 generator 시작 모듈.
GeneratorStage	각 해상도에서 upsampling, channel mapping, 여러 개의 R3ResidualBlock 을 수행하는 stage.
to_rgb	각 해상도 feature map 을 3 채널 RGB 이미지로 변환하는 1×1 convolution.
transition lerp	이전 해상도 출력을 upsample 한 이미지와 현재 해상도 출력을 alpha 로 선형 결합.

Discriminator 아키텍처 (**Image** $\rightarrow$ **from_rgb** $\rightarrow$ **Progressive Down Stages** $\rightarrow$ **DiscriminativeBasis** $\rightarrow$ **Score**)

from_rgb	입력 RGB 이미지를 현재 해상도의 feature map 으로 변환하는 1×1 convolution.
DiscriminatorDownStage	R3ResidualBlock 이후 fixed-filter downsampling 으로 해상도를 줄이는 stage.
transition lerp	현재 해상도 경로와 이전 해상도로 downsample 한 경로를 alpha 로 선형 결합한다.
DiscriminativeBasis	최종 4×4 feature map 을 하나의 scalar score logit 으로 변환한다.



2. 모델 설계 아이디어

2.1 베이스라인 선정: StyleGAN2 대신 R3GAN 구조를 채택한 이유

본 프로젝트는 NeurIPS 2024 논문 The GAN is dead; long live the GAN! A Modern GAN Baseline 의 R3GAN 을 baseline 으로 선정하였다. R3GAN 은 StyleGAN2 의 복잡한 구성 요소를 줄이고, ResNext-style backbone 과 regularized relativistic loss 를 사용해 단순하면서도 강한 GAN baseline 을 제안한다. 본 프로젝트에서는 grouped convolution 기반 residual block, generator/discriminator, relativistic softplus loss, real/fake 양쪽 R1/R2 gradient penalty 를 직접 구현하였다.

2.2 R1, R2 Gradient Penalty 의 개념과 역할

R3GAN 은 discriminator 안정화를 위해 R1/R2 zero-centered gradient penalty 를 사용한다. R1 은 real image, R2 는 generated image 에 대한 discriminator score 의 입력 gradient norm 을 억제하는 항이다. 이는 discriminator 가 real/fake sample 주변에서 지나치게 sharp 한 판별 경계를 만드는 것을 막고, relativistic loss 학습에서 발생할 수 있는 mode dropping 과 non-convergence 를 줄이는 역할을 한다.

2.3 R3GAN Regularization 과 Exact GP 최적화

R1/R2 gradient penalty 는 일반적인 PyTorch 구현에서 create_graph=True 와 double backward 를 필요로 하며, 이는 고차 미분 그래프 유지로 인해 학습 속도와 메모리 측면에서 부담이 크다. 본 프로젝트는 A Closer Look at Double Backpropagation 논문을 참고하여, discriminator 의 piecewise-linear 구조를 활용한 exact GP 경로를 구현하였다. loss.py 에서는 BiasLeakyReLU 의 activation mask 를 기록하고, backward 단계에서 convolution/residual block 의 전치 연산을 직접 적용하여 입력 gradient 와 parameter gradient 를 계산한다. 이를 통해 PyTorch 의 일반적인 double backward 에 의존하지 않고 R1/R2 regularization 을 유지하였다.

2.4 Progressive Generation 파이프라인

고해상도 학습을 안정화하기 위해 $64 \times 64 \rightarrow 128 \times 128 \rightarrow 256 \times 256 \rightarrow 512 \times 512$ 로 해상도를 확장하는 progressive training pipeline 을 설계하였다. 코드 구조상 1024×1024 stage 도 지원하지만, 최종 제출 모델은 안정성과 품질을 고려해 512×512 checkpoint 를 사용하였고, 제출용 1024×1024 출력은 inference/export 단계에서 upsampling 으로 맞추었다.

train_wrapper.py 는 stage 별 config 를 하나의 pipeline config 로 묶어 순차 실행하고, 이전 stage checkpoint 를 다음 stage 의 초기화 및 teacher checkpoint 로 자동 연결한다. 또한 train.py 의 transition 모드에서는 이전 해상도 출력을 upsample 한 결과와 현재 해상도 출력을 alpha 로 선형 결합하고, transition 단계에서만 low-resolution consistency loss 를 적용한다. 이를 통해 해상도 전환 시 출력 분포가 급격히 변하는 문제를 완화하였다.

3. 최종 제출 모델 학습 레시피

3.1 config 설정 (Stage 공통항목)

[항목]	[값]	[항목]	[값]
Generator Widths	[768, 768, 768, 768, 512, 256, 128, 96, 96]		
Discriminator Widths	[768, 768, 768, 768, 512, 256, 192, 128, 96]		
Generator Cardinality	[96, 96, 96, 96, 64, 32, 16, 12, 12]		
Discriminator Cardinality	[96, 96, 96, 96, 64, 32, 24, 16, 12]		
R3_gamma (R1/R2 penalty 강도)	5		
ema_beta	0.999		
Amp dtype (mixed precision dtype)	BF16		
Generator LR	0.0002	Discriminator LR	0.0002
Beta1(Adam optimizer momentum)	0	Beta2	0.99
Fid.num_real_images	1024	Fid.num_fake_images	1024

3.2 Stage 별 config 설정 (학습 전 과정은 Colab 의 A100 GPU 80GB VRAM 환경에서 수행)

[항목]	64_base	128_tran	128_stab	256_tran	256_stab	512_tran	512_stab
progressive.resolution	64	128	128	256	256	512	512
progressive.stage_mode	stabilize	transition	stabilize	transition	stabilize	transition	stabilize
progressive.alpha.start->end	X	0->1	1	0->1	1	0->1	1
~.alpha. duration_kimg	X	100	X	100	X	64	X
training.init_from	False	True	True	True	True	True	True
training.consistency	False	True	False	True	False	True	False
~.consistency.start -> end	X	0.3->0	X	0.3->0	X	0.2->0	X
~.consistency.duration_kimg	X	100	X	100	X	64	X
data.batch_size	64	64	64	24	24	12	12
training.grad_accum_steps	1	1	1	2	2	2	2
training.total_steps	15,000	5,000	10,000	5,000	10,000	5,000	10,000
training.fid.interval	2,500	X	2,500	X	2,500	X	2,500

Progressive.alpha: transition 에서 이전 해상도 출력과 현재 해상도 출력을 섞는 비율로 duration_kimg 가 되는 시점까지 start 에서 end 까지 선형으로 증가한다.

Training.consistency: transition 에서 teacher 모델과 low-resolution consistency 를 맞추는 보조 loss 설정. 전환 초반 구조 붕괴를 줄이는 역할로 duration_kimg 가 되는 시점까지 start 에서 end 까지 선형으로 감소한다.

training.grad_accum_steps: gradient accumulation 횟수. $\text{batch_size} \times \text{grad_accum_steps} = \text{effective batch}$.

4. 모델 비교 및 결과 분석

4.1 비교 모델 설명

R3GAN 은 StyleGAN2 의 복잡한 구성 요소를 줄이고, 직관적인 구조와 zero-centered gradient penalty 를 통해 안정적인 학습을 목표로 하는 모델이다. 이에 따라 본 프로젝트의 개선도 부가적인 ablation 을 누적하기보다, 256 해상도에서 가능성을 확인한 뒤 gradient penalty 계산 효율화와 progressive 고해상도 학습 안정화에 집중하였다. 최종적으로는 코드 구조가 완성된 V1 과, channel 확장 및 effective batch 조정을 적용한 제출 버전 V2 를 비교하여 성능과 한계를 분석하였다.

4.2 V1 의 차이점

Generator, Discriminator Widths [768, 768, 768, 768, 384, 192, 96, 64]

Generator, Discriminator Cardinality [96, 96, 96, 96, 48, 24, 12, 8]

V1	64_base	128_tran	128_stab	256_tran	256_stab	512_tran	512_stab
progressive.alpha.start->end	X	0->1	1	0->1	1	0->1	1
~.alpha.duration_kimg	X	100	X	100	X	50	X
training.consistency	False	True	False	True	False	True	False
~.consistency.start -> end	X	1->0	X	0.5->0	X	0.25->0	X
~.consistency.duration_kimg	X	100	X	100	X	50	X
data.batch_size	64	64	64	24	24	16	16
training.grad_accum_steps	1	1	1	1	1	1	1
training.total_steps	8,000	2,000	5,000	5,000	10,000	4,000	20,000

4.3 V1 512_stabilize 의 training sample 및 FID 분석

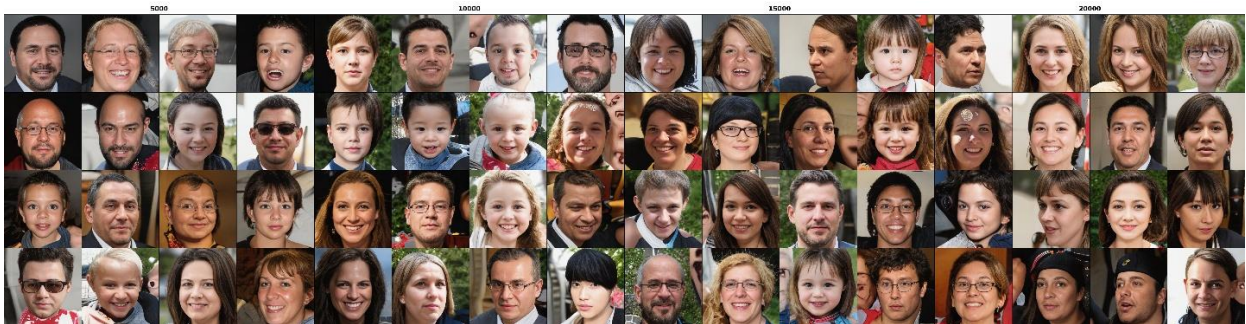


그림 2. V1 512_stabilize EMA 의 random noise sample (순서대로 5k, 10k, 15k, 20k)

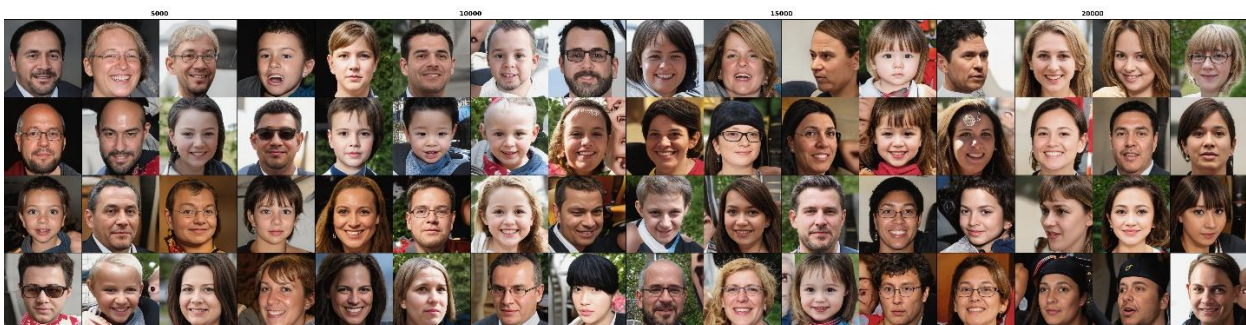


그림 3. V1 512_stabilize EMA 의 fixed noise sample (순서대로 5k, 10k, 15k, 20k)

그림 2 와 그림 3 을 보면 sample 이미지가 training 이 진행되어도 더 이상 개선되고 있지 못하고 측면을 바라보는 얼굴 구조의 무너짐, 배경 무너짐 같은 구조적인 문제가 반복되고 있다. 또한 1k validation set 으로 계산한 FID 값 또한 47.63->49.89->47.94->47.48 로 더 이상 내려가지 않았다. 따라서 512 stage 의 문제는 단순 학습량 부족보다는 모델 용량과 batch 안정성의 한계로 판단하였다.

이를 바탕으로 V2 에서는 generator/discriminator channel 폭을 확장하고, gradient accumulation 을 통해 effective batch 를 늘렸다. 또한 cuDNN benchmark 를 비활성화하여 VRAM peak 를 낮춤으로써 512 stage 를 더 안정적으로 장시간 학습할 수 있도록 하였다.

4.4 V2 512_stabilize 의 training sample 및 FID 분석



그림 4. V2 512_stabilize EMA 의 random noise sample (순서대로 1k, 5k, 10k)



그림 5. V2 512_stabilize EMA 의 fixed noise sample (순서대로 1k, 5k, 10k)

그림 4 와 그림 5 를 보면 channel 폭을 확장, effective batch 증가, 낮은 해상도에서 학습 kimg 증가를 했음에도 v1 과 동일한 문제가 반복되고 있음을 확인할 수 있다. 그림 5 의 fixed noise sample 은 심지어 학습이 더 진행될수록 구조 붕괴가 심해지는 현상이 있어 학습을 10k 시점에서 중단했다. 하지만 1k FID 의 값은 43.76(2,500)->43.40(5,000)->42.11(7,500)->41.38(10,000)으로 v1 보다 낮게 측정되었고, 제출 결과는 FID 가 19.313 으로 준수하게 나왔다. 따라서 V2 는 최종 제출 모델로는 충분한 성능을 보였지만, 고해상도 세부 영역의 정교함에는 개선 여지가 남았다.

5. W&B 그래프 및 로그

5.1 V2 모델 512_stabilize 모니터링 및 학습 곡선



그림 6. W&B Charts

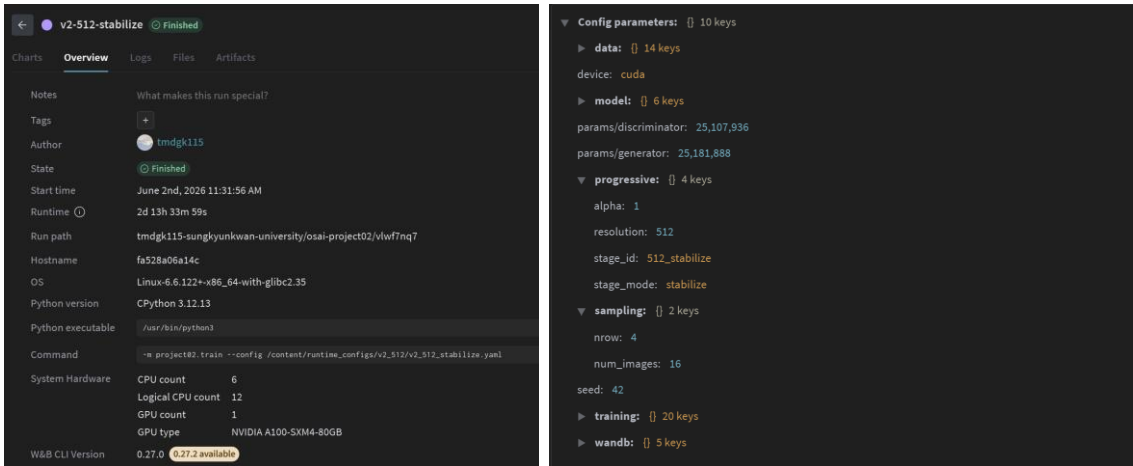


그림 7, 8. W&B Overview · Config 페이지 — 모델, 데이터, 시드, 하드웨어 정보 (training 정보는 학습 레시피에 존재)

5.2 AI 도구 사용

AI 도구는 프로젝트 수행 과정에서 보조 도구로 적극적으로 활용하였다. 작업 세션을 분석, 기획, 검토, 구현, 버그 확인, 실험 설계 등으로 분리하여 각 단계별 산출물을 만들고, 이를 바탕으로 코드 수정 방향과 실험 우선순위를 정리하였다. 또한 Liner AI 등을 활용해 관련 논문과 구현 아이디어를 탐색하였고, 보고서 작성 과정에서도 문장 정리와 구성 보조에 활용하였다. 단, 최종 코드 반영 여부와 실험 결과 해석은 직접 검토하여 결정하였다.

참고문헌

- [1] N. Huang, A. Gokaslan, V. Kuleshov, and J. Tompkin, "The GAN is dead; long live the GAN! A Modern GAN Baseline," NeurIPS, 2024. arXiv:2501.05441.
- [2] C. Etmann, "A Closer Look at Double Backpropagation," arXiv preprint, 2019. arXiv:1906.06637.